

Министерство цифрового развития, связи
и массовых коммуникаций Российской Федерации

Сибирский государственный университет
телекоммуникаций и информатики

Кафедра прикладной математики и кибернетики

Расчетно-графическая работа
По дисциплине: «Защита информации»
Вариант 3

Выполнили:

Студент 4 курса группы ИП-111

Попов М.И.

Проверил:

Старший преподаватель ПМиК

Дьячкова И.С

Новосибирск, 2024

Постановка задачи

Необходимо написать программу, реализующую протокол доказательства с нулевым знанием Фиата-Шамира.

Для выполнения этого варианта задания необходимо разработать клиент серверное приложение с авторизацией по протоколу Фиата-Шамира. Открытые ключи с соответствующими логинами должны храниться в файле (или базе данных) на сервере, клиентское приложение при этом не должно отправлять на сервер никаких закрытых данных, закрытый ключ нигде не хранится и используется исключительно для осуществления работы протокола с клиентской стороны. Все открытые параметры системы рассылаются сервером при установке соединения с клиентом.

Программа должна наглядно демонстрировать работу алгоритма, возможно (но не обязательно) в графическом режиме. Текст программы должен содержать исчерпывающие комментарии, тем не менее, следует воздержаться от описания очевидных действий. К РГР необходимо представить отчёт, оформленный в соответствии с требованиями, предъявляемыми к работам подобного типа. В отчёт должны быть включены в обязательном порядке: Титульный лист, содержание, постановка задачи, исходный код с комментариями, скриншоты с результатами работы программы.

Теоретические сведения

Одним из наиболее известных протоколов идентификации личности с помощью доказательства с нулевым знанием является протокол, предложенный Амосом Фиатом и Ади Шамиром, стойкость которого основывается на сложности извлечения квадратного корня по модулю достаточно большого составного числа n , факторизация которого неизвестна.

Предварительно, перед самым доказательством доверенный центр T выбирает и публикует модуль достаточно большого числа $n = p \cdot q$, разложить на множители которое трудно. При этом p , q – простые числа и держатся в секрете. Каждый пользователь A выбирает секретное s из интервала $(1, n-1)$ взаимно простое с n . Затем вычисляется открытый ключ $v = s^2 \pmod{n}$.

Полученное v регистрируется центром доверия в качестве открытого ключа пользователя A , а значение s является секретом A . Именно знание этого секрета s необходимо доказать A стороне B без его разглашение за t раундов. Каждая аккредитация состоит из следующих этапов:

1. A выбирает случайное r из интервала $(1, n-1)$ и отправляет $x = r^2 \pmod{n}$ стороне B .
2. B случайно выбирает бит e (0 или 1) и отправляет его A .
3. A вычисляет $y = r \cdot s^e \pmod{n}$ и отправляет его обратно к B .
4. Сторона B проверяет равенство $y^2 \equiv x \cdot v^e \pmod{n}$. Если оно верно, то происходит переход к следующему раунду протокола, иначе доказательство не принимается.

Выбор e из множества предполагает, что если сторона A действительно знает секрет, то она всегда сможет правильно ответить, вне зависимости от выбранного e . Допустим, что A хочет обмануть B , выбирает случайное r и отправляет $x = r^2 \pmod{n}$, тогда если $e=0$, то A удачно возвращает B $y = r$, в случае же $e=1$, A не сможет правильно ответить, т.к. не знает s , а извлечь квадратный корень из v по модулю n достаточно сложно.

Вероятность того, что пользователь A не знает секрета s , но убеждает в обратном проверяющего B будет оцениваться вероятностью равной $p = 2^{-(t)}$, где t – число аккредитаций. Для достижения высокой достоверности его выбирают достаточно большим ($t = 20 - 40$). Таким образом, B удостоверяется в знании A тогда и только тогда, когда все t раундов прошли успешно.

Для того чтобы этот протокол корректно выполнялся, сторона A никогда не должна повторно использовать значение x . Если бы A поступил таким образом, а B во время другого цикла отправил бы A на шаге 2 другой случайный бит r , то B бы имел оба ответа A . После этого B может вычислить значение s , и ему будет известен секретный ключ Алисы.

Результат работы программы

```
PS G:\_sibsutis\ZI> python.exe .\server.py
```

```
□ Сервер запущен
```

```
p =
```

```
12189241989457472843996883194587756264747662120144516291734138597485999
06832932009469011539948224965918013562059185954124098988160442695282063
83340404657444016451673010938099675899786266740568407366003521183484779
97440072324039387093253366215954037127449466674401634454654541493017674
7150233120862572830106259
```

```
q =
```

```
17172931723987081683539300933358993595949519075574164207184607005640634
51266563188794197414523996765653131982515401539579882299783556433230364
66041172196351179946588310927204895800943774114316567469545834643073434
43316790212304191261797328421229335648516897315555645311031725230560933
7131733448014137037486723
```

```
self.n =
```

```
20932502045210964446610672565909395245902910297571007566718197265694847
59105440914175787211270394246893160636989087443516104163393545819817625
78700995400157742468996070970247000374947757432713460869294693112279750
91272000181212953769972679031311908100891372919044585318630308041341979
73270094102663497594701067596771499197440002258426698529115279193573390
05054189884183170969830349251513861107533902256890471519720840771544136
15394508019109551705954463266676892354802052153889377347726047106006760
71456913088697952628476665630787384653864166222058863958054817612030054
1040064979700345812692761283751032933002391699257
```

```
*****
```

```
!Не удалось открыть файл users.json, создаю новый
```

```
□ Сервер функционирует
```

□ Совершена попытка подключения: ('127.0.0.1', 63309)

Клиент ('127.0.0.1', 63309) отправлено n

□ Клиент ('127.0.0.1', 63309) зарегистрировался с именем 'Alise' и ключом v =
10952989733127771417412353385439774456542667397166376897646481566538228
23516198959912626244076985660265690219474814038602072278531097160194565
47582814725890691323422623846131714537343322957003213774091202079170077
72343206492087769243749569371344782235000163328509815142746562555123188
04854220649261713554501624344421724284041241744341680827178012264315361
31499357217670757662435875593924373657405820150259847329684097530134781
27224188918204059772728396906626299148937983525136948656718851041472414
56624031031424038887187776084902461075552353265396206206733577350509239
4632996156333548040471030520166464534574066653612

□ Соединение с клиентом потеряно ('127.0.0.1', 63309)

□ Совершена попытка подключения: ('127.0.0.1', 63310)

Клиент ('127.0.0.1', 63310) отправлено n

□ Клиент ('127.0.0.1', 63310) пытается авторизоваться под именем 'Alise'

1.

x =

20237507282991198497836791583013992995780258222533481712899614546206715
06298180712011883376038432806188580444990075654497594787341563172302355
28638135888884728048742068369016097152170001330325727708473322652979866
28744404864079245788819024958904155910518619837729632704067092085526802
19145003072443145922088944439332362289182426706856924421229132376394383
85375699125993062668090667617608200430252584505137455211030980123107913
25523287132543566951949399571613314239279101562536106671691822712772746
12504493217334450571869140117448219372981327189635776685989251922787736
1668203087232182329904844152396268243668741368652

e = 1

	y	=
10984221668007271752149220364319027862277969826533235476233673746951686		
67719595527969433770657068541160219010233371917691739610340446811497678		
26607752983656963748332107947172901212834896203259068831736681225995554		
69578327576350742369925838718740504329336838760499623901035549341001278		
46648998455557110663074400077014519621720182462762782293110010846757752		
70627173724851669749491589114019353851602164808202600126577385590821268		
74523709808171587961297106028449312860350342162820631461405092818016582		
59301606753505668155435258392810427653961116121953245390761897979537528		
9603821454852110978767628625114913476800920308665		
--		
	y^2	% n =
17798542719830432871018176684464180485197179471225455189337560848145110		
58033917172201892726101153746167876470496114472972580539773962176412234		
28251682007154714588322909339373430610233642715007045056703038407429323		
83732061480524140307805295581775992836253929793040331953458743999506213		
54140536127693630091300515668298768450660826373591150773320816403055449		
16158021464888630353017456022623000527804992445962493774133736030557599		
74978336857266658912654533786489008945394505797192948563879250627180962		
36037400139708192556853264132871670784248851196996625753863310575023456		
9461745825646068218781402198827881937296360143373		
	x	* v^e % n =
17798542719830432871018176684464180485197179471225455189337560848145110		
58033917172201892726101153746167876470496114472972580539773962176412234		
28251682007154714588322909339373430610233642715007045056703038407429323		
83732061480524140307805295581775992836253929793040331953458743999506213		
54140536127693630091300515668298768450660826373591150773320816403055449		
16158021464888630353017456022623000527804992445962493774133736030557599		
74978336857266658912654533786489008945394505797192948563879250627180962		
36037400139708192556853264132871670784248851196996625753863310575023456		

9461745825646068218781402198827881937296360143373

(success) $y^2 == x * v^e \% n$

□ Аутентификация 'Alise' пройдена успешно

□ Соединение с клиентом потеряно ('127.0.0.1', 63310)

PS G:_sibsutis\ZI> python.exe .\client.py

□ Попытка соединения с сервером для регистрации юзера с именем 'Alise'...

□ Соединение с сервером установлено

□ С сервера получено n

□ Пользователь успешно зарегистрирован

□ Разорвано соединение с сервером

□ Попытка соединения с сервером для авторизации пользователя под именем 'Alise'...

□ Соединение с сервером успешно

□ С сервера получено n

□ Авторизация пройдена успешно

□ Соединение с сервером разорвано

{"Alise":

50248255951484041557968260991616748637037931579996528224635546009523089
87201419468459312111848121297056526893065086091709492897801972776518929
42573354661630445611853845542420923874500147190025382251581227089193799
43205273413631520887232182452915040686785679588201422314354605455594649
10948624118573224109797240307943764986449665001624992462707354188693634
57074828717967960542193390866693487855679040150380918450846494748779950
93983012519062072221025056106529143584792147218349048798176033515732439
52916778914948291310702141555548289664478584315427608507740853285059892
998394252025679415212510159433514332066530128170}

Клиент генерирует секрет s , который взаимно прост с n (произведением двух больших простых чисел p и q , сгенерированных сервером). Клиент вычисляет открытый ключ $v = s^2 \bmod n$ и отправляет его серверу вместе со своим именем. Сервер сохраняет v как связанный с именем пользователя идентификатор. Этот процесс не требует передачи секретного s , поэтому секрет остается неизвестным серверу.

Листинг

```
import math
import random
import sys

def exponentiation_modulo(a: int, x: int, p: int) -> int:
    if p == 0:
        raise ValueError("Модуль не может быть равен нулю")
    if x < 0:
        raise ValueError("Показатель не может быть отрицательным")
    result = 1
    a = a % p
    if a == 0:
        return 0
    while x > 0:
        if x & 1 == 1:
            result = (result * a) % p
        a = (a ** 2) % p
        x >>= 1
    return result

def is_prime(p: int, trials: int=20) -> bool:
    if p == 2 or p == 3:
        return True
    if p < 2 or not (p & 1):
        return False
    for _ in range(trials):
        a = random.randint(2, p - 1)
        if exponentiation_modulo(a, (p - 1), p) != 1 or math.gcd(p, a) > 1:
            return False
    return True

def gen_prime(left: int, right: int) -> int:
    while True:
        p = random.randint(left, right)
        if is_prime(p):
            return p

def gen_safe_prime(a: int, b: int) -> int:
    if a > b:
        a, b = b, a
    while True:
        q = gen_prime(a // 2, (b - 1) // 2)
```

```

    if is_prime(p := q * 2 + 1):
        return p

def gen_g(p: int) -> int:
    while True:
        g = random.randrange(2, p)
        if pow(g, (p - 1) // 2, p) != 1:
            return g

def gen_mutually_prime(p: int):
    while True:
        if math.gcd(p, b := random.randrange(2, p)) == 1:
            return b

def gen_mutually_prime_big(p: int):
    while True:
        if math.gcd(p, b := random.randrange(p // 2, p)) == 1:
            return b

def generalized_euclidean_algorithm(a: int, b: int) -> list[int, int, int]:

    if a <= 0 or b <= 0:
        raise ValueError("Числа могут быть только натуральными")
    if a > b:
        a, b = b, a
    u = [a, 1, 0]
    v = [b, 0, 1]
    while v[0] != 0:
        q = u[0] // v[0]
        t = [u[0] % v[0], u[1] - q * v[1], u[2] - q * v[2]]
        u, v = v, t
    return u

def inverse(n, p):
    gcd, inv, _ = generalized_euclidean_algorithm(n, p)
    assert gcd == 1
    if inv < 0:
        inv += p
    return inv

```

Листинг 1 – все логические функции используемы для шифрования (logic.py)

```

import json
from logic import *
import socket
import signal

t = 1

class Server:
    def __init__(self, host: str = "localhost", port: int = 3000):

```

```

self._host = host
self._port = port
self._buffer_size = 1024

p = q = gen_prime(1 << 1023, (1 << 1024) - 1)
while p == q:
    q = gen_prime(1 << 1023, (1 << 1024) - 1)

self.n = p * q

print(f"\t{p = }", f"\t{q = }", f"\t{self.n = }", '*' * 30, sep='\n')

try:
    with open("users.json", "r") as f:
        self._registered_users = json.load(f)
    print("✓ Файл users.json найден!")
except FileNotFoundError:
    with open("users.json", "w") as f:
        self._registered_users = dict()
        json.dump(self._registered_users, f)
    print(
        "!Не удалось открыть файл users.json, создаю новый")

def run(self):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.bind((self._host, self._port))
    sock.listen()
    print("☐ Сервер функционирует")

    while True:
        conn, addr = sock.accept()
        print(f"\n☐ Совершена попытка подключения: {addr}")

        conn.send((bytes(str(self.n), encoding = "utf8")))

        print(f"Клиент {addr} отправлено n")

        data = conn.recv(self._buffer_size).decode("utf8")

        if data == "{register}":
            name = conn.recv(self._buffer_size).decode("utf8")
            v = int(conn.recv(self._buffer_size).decode("utf8"))

            if name not in self._registered_users.keys():
                print(f"☐ Клиент {addr} зарегистрировался с именем '{name}' и ключом v = {v}")
                self._registered_users[name] = v
                with open("users.json", "w") as f:
                    json.dump(self._registered_users, f)
                conn.send((bytes("success", encoding = "utf8")))
            else:

```

```

        print(f"❌ Имя '{name}' уже зарегистрировано")
        conn.send((bytes("already registered", encoding = "utf8")))

    if data == "{auth}":
        name = conn.recv(self._buffer_size).decode("utf8")
        v = self._registered_users[name]

        print(f"❌ Клиент {addr} пытается авторизоваться под именем '{name}'")

        for i, _ in enumerate(range(t), 1):
            x = int(conn.recv(self._buffer_size).decode("utf8"))

            e = random.randint(0, 1)
            conn.send((bytes(str(e), encoding = "utf8")))

            y = int(conn.recv(self._buffer_size).decode("utf8"))

            y2 = exponentiation_modulo(y, 2, self.n)
            xv = x * exponentiation_modulo(v, e, self.n) % self.n
            print(f"{i}. \n\t{x = } \n\t{e = } \n\t{y = } \n\t-- \n\t{y^2 % n = {y2}} \n\t{x * v^e % n = {xv}}")

            if y2 == xv:
                print(f"(success) y^2 == x * v^e % n\n")
                if i == t:
                    print(f"Аутентификация '{name}' пройдена успешно")
                    conn.send((bytes("success", encoding = "utf8")))
                else:
                    conn.send((bytes("check", encoding = "utf8")))
            else:
                print(f"(fail) y^2 != x * v^e % n\n")
                conn.send((bytes("fail", encoding = "utf8")))
                print(f"❌ Аутентификация '{name}' не пройдена")
                break

        conn.close()
        print(f"❌ Соединение с клиентом потеряно {addr}")

if __name__ == "__main__":
    print("❌ Сервер запущен")
    signal.signal(signal.SIGINT, signal.SIG_DFL)
    server = Server()
    server.run()

```

Листинг 2 – Сервер (server.py)

```

import random

from logic import *
import socket

```

```

random.seed(1)

class Client:
    def __init__(self, name: str):
        self.name = name
        self._buffer_size = 1024

    def register(self, host: str = "localhost", port: int = 3000):
        print(f"Попытка соединения с сервером для регистрации юзера с именем '{self.name}'...")
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.connect((host, port))
        print(f"Соединение с сервером установлено")
        n = int(sock.recv(self._buffer_size).decode("utf8"))
        print(f"С сервера получено n")

        sock.send((bytes("{register}", encoding = "utf8")))
        sock.send((bytes(self.name, encoding = "utf8")))

        self.s = gen_mutually_prime_big(n)
        v = exponentiation_modulo(self.s, 2, n)

        sock.send((bytes(str(v), encoding = "utf8")))

        status = sock.recv(self._buffer_size).decode("utf8")
        if status == "success":
            print(f"Пользователь успешно зарегистрирован")
        elif status == "already registered":
            print(f"Пользователь под именем '{self.name}' уже зарегистрирован")

        print(f"Разорвано соединение с сервером\n")
        sock.close()

    def auth(self, host: str = "localhost", port: int = 3000):
        print(f"Попытка соединения с сервером для авторизации пользователя под именем '{self.name}'...")
        sock = socket.socket()
        sock.connect((host, port))
        print(f"Соединение с сервером успешно")

        sock.send((bytes("{auth}", encoding = "utf8")))

        n = int(sock.recv(self._buffer_size).decode("utf8"))
        print(f"С сервера получено n")

        sock.send((bytes(self.name, encoding = "utf8")))

        while True:
            r = random.randrange(1, n - 1)
            x = exponentiation_modulo(r, 2, n)

```

```

sock.send((bytes(str(x), encoding = "utf8")))

e = int(sock.recv(self._buffer_size).decode("utf8"))

y = r * self.s ** e % n

sock.send((bytes(str(y), encoding = "utf8")))

status = sock.recv(self._buffer_size).decode("utf8")
if status == "success":
    print("□ Авторизация пройдена успешно")
    break
elif status == "fail":
    print("□ Отказ в авторизации")
    break
elif status == "check":
    pass

print(f"□ Соединение с сервером разорвано\n")
sock.close()

if __name__ == '__main__':

    alice = Client('Alise')
    alice.register()
    alice.auth()

    #fake = Client('Alise')
    #fake.s = 228
    #fake.auth()

```

Листинг 3 - Клиент (client.py)